CS & IT ENGINEERING



Algorithms

Sorting Algorithms



Lecture No.- 03

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Bubble Sort

Selection Sort

Topics to be Covered



- Topic
- Topic
- Topic

Insertion Sort

Radix Sort

Questions + PYQs



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



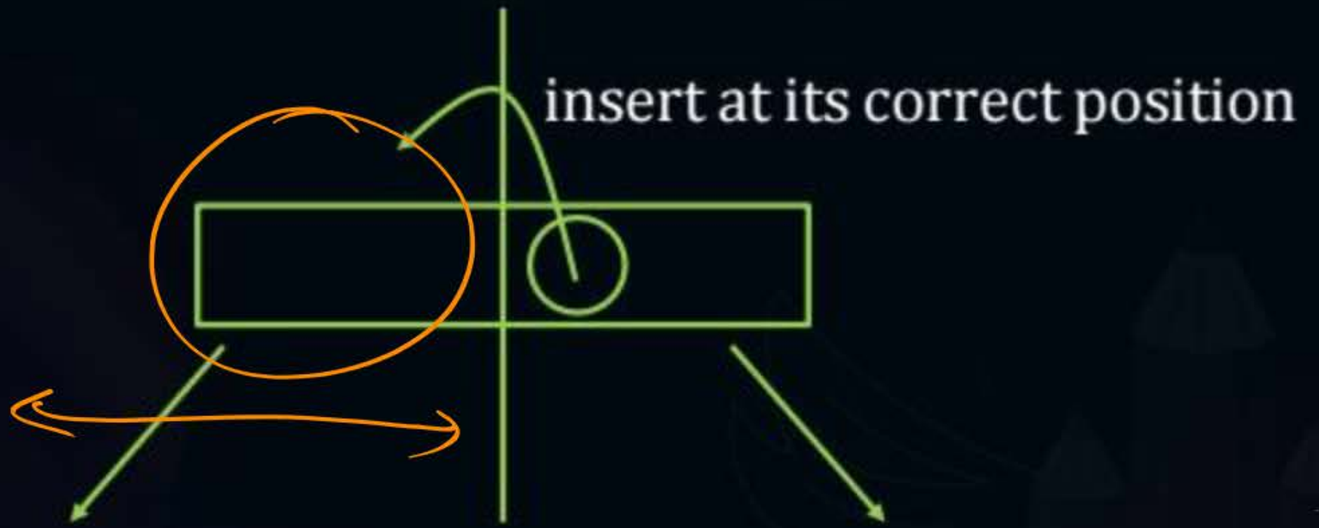
Telegram Link for Aditya Jain sir:

https://t.me/AdityaSir_PW



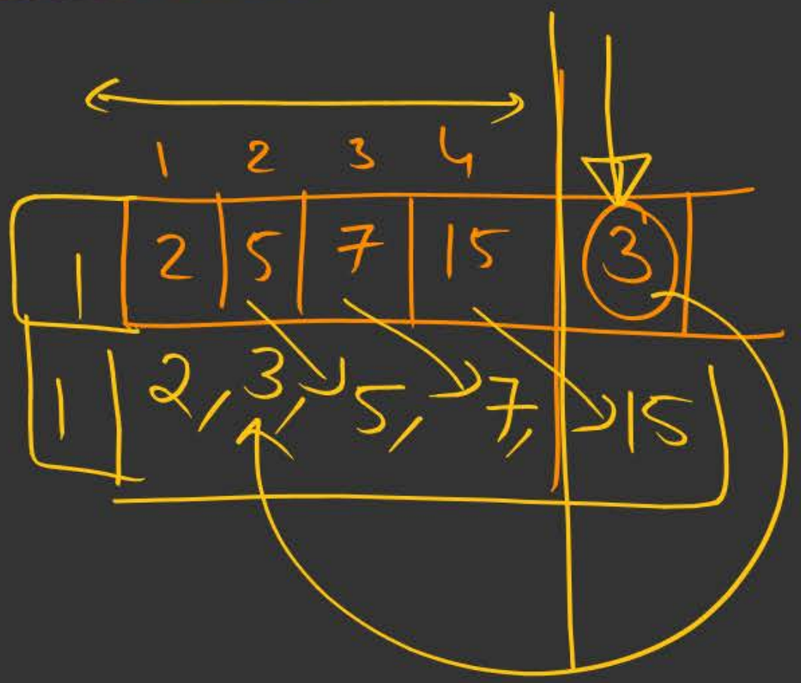
Topic : Sorting Algorithms

Logic: Insertion Sort



How insertion is done ?





inversion pair





Topic : Sorting Algorithms

Code Walkthrough:

85	22	49	97	86	15

Pass1: 85 | 22 49 97 86 15

Pass2: 22 85 | 49 97 86 15

Pass3: 22 49 85 | 97 86 15

Pass4: 22 49 85 97 | 86 15

Pass5: 22 49 85 86 97 | 15

(15 22 49 85 86 97) Sorted

B.C pass

W.C pass

85

Key → to be inserted in sorted part.

Topic : Sorting Algorithms

Insertion Sort Code: 1 ~~bonded~~ based indexing

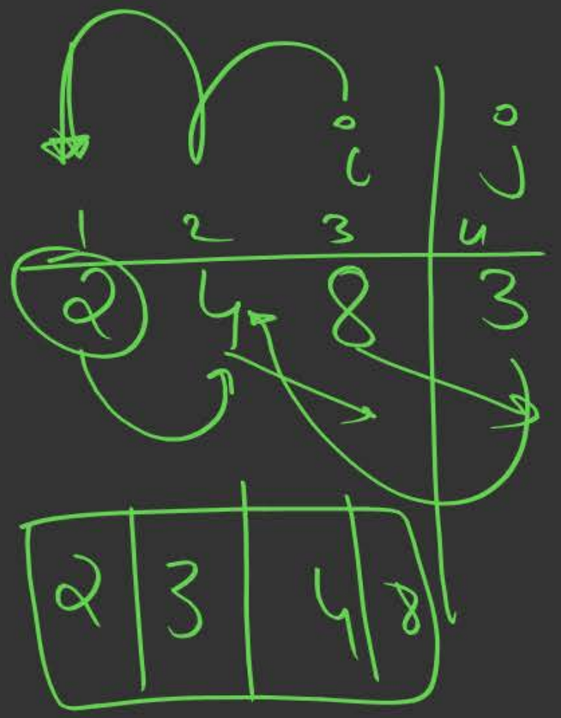
```
Algo InsertionSort(A, n)
{
    for (j = 2; j <= n; j++)
    {
        key = A[j]
        i = j – 1
        while (i > 0 and A[i] > key) {
            A[i + 1] = A[i]
            i --
        }
        A[i + 1] = key
    }
}
```





shift ≈ interchange (swap)





$i = 3 \not> 1$

exit

$i = 1$

$A[i+1] = key$



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

i/p : A = 80 | 20 40 90 30 60

P1: j = 2;

key = A[j] = A[2] = 20

i = j – 1 = 2 – 1 = 1

while (i > 0 and A[i] > key)

{

A[i + 1] = A[i]

}

A[0 + 1] = 20

A[2] = A[1]

i = 0



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

A = 20 80 | 40 90 30 60

P2: j = 3;

key = A[3] = 40

i = j – 1 = 3 – 1 = 2

A[1 + 1] = 40



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

A = 20 40 80 | 90 30 60

P3: j = 4;

key = A[4] = 90

i = j – 1 = 4 – 1 = 3

i = 3

A[3 + 1] = key = 90

A[4] = 90

This is not a swap



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

A = 20 40 80 90 | 30 60

P4: j = 5; 30 40 80 90

key = A[2] = key = 30

i = j – 1 = 5 – 1 = 4 3 2 1

A[1+1] = 30

A[2] = 30



Topic : Sorting Algorithms

GATE-25

MSQ

Dry-Run of Previous Insertion Sort Code:

A = 20 30 40 80 90 | 60

P5: j = 6;

60 > 80 × 40

key = A[4] = key = 60

~~i = j - 1 = 5 - 1 = 3~~

i = j - 1 = 6 - 1 = 5

i = ~~5~~ 4 3

A[3+1] = 60

[20 30 40 60 80 90] → Sorted

Best Case :- i/p → already Sorted in Inc order



B.C TC = O(n+0)
= O(n) ⇒ Ω(n)

Every pass = 1 Comp
Every pass = 0 swaps
Total passes = (n-1) passes

Total comp = 1*(n-1)
= (n-1)
= 0 swaps



Topic : Sorting Algorithms

Time Complexity Analysis:

Worst Case:

i/p is sorted in opposite order

A = 80, 70, 65, 40, 10

Pass1 → 1 comp, 1 swap

Pass2 → 2 comp, 2 swap

Pass3 → 3 comp, 3 swap

.

.

Pass(n–1) → (n–1) comp, (n–1) swap

Total in all (n–1) passes

No. of comp

$$= 1 + 2 + 3 \ldots (n-1) = \frac{n(n-1)}{2}$$

No. of swaps

$$= 1 + 2 + 3 \ldots (n-1) = \frac{n(n-1)}{2}$$



Topic : Sorting Algorithms

Summary:

	No. of Comparisons	No. of Swaps (inversions)
Best case increasing order	$(n-1)$ +	0 → $\Omega(n)$
Worst Case Descending order	$\frac{n(n-1)}{2}$ +	$\frac{n(n-1)}{2}$ → $O(n^2)$

Space Complexity → $O(1)$ → Inplace ✓



Topic : Sorting Algorithms

Important Points on Insertion Sort:

1. Always takes (n – 1) passes
2. Time complexity

   BC → $\Omega(n)$

   WC → $O(n^2)$

   $A(i) > A(j)$
3. Space Complexity → O(1) → Inplace
4. Also stable
5. If the input list is pre-sorted, then it takes time of O(n + d),

   where, n = no. of elements

   d = no. of inversions

→ Question



Topic : Sorting Algorithms

Note:

We use this while solving questions when the input sequence/order is not known and only no. of inversions are mentioned.

n elem

no. of inv = O(n)

T.C ?

↳ TC : O(n+d)

= O(n+n) = O(n)

(8) A: [9 3 5 10 7 2 4 15 1]

using Insertion Sort:

no. of Comp = x

no of swaps = y

then x+y = ?

A = 9 3 5 10 7 2 4 15 1

pass1: → 9 | 3

pass2: 3 9 | 5

pass3: 3 5 9 | 10

pass4: 3 5 9 10 | 7

pass5: 3 5 7 9 10 | 2

pass6: 2 3 5 7 9 10 | 4

pass7: 2 3 4 5 7 9 10 | 15

pass8: 2 3 4 5 7 9 10 15

[1 2 3 4 5 7 9 10 15]

Comp	Swaps
1	1
2	1
1	0
3	2
5	5
5	4
1	0
8	8
26	

Comp = 26 = x

Swaps = 21 = y

Ans:- 47

13:1



Topic : Sorting Algorithms

Radix Sort (Non-Comparison based)

- No. of passes = no. of digits in the max element of given input
- Makes use of Buckets/Bins



Topic : Sorting Algorithms

Example: Base = 10 (decimal)

i/p: A = 64, 723, 99, 83, 545, 65, 333, 7, 4, 124

Input : {L → R}

o/p : Buckets {L → R, Bottom to Top}

max (A) = 723

no. of digits (723) = 3 → means 3 passes



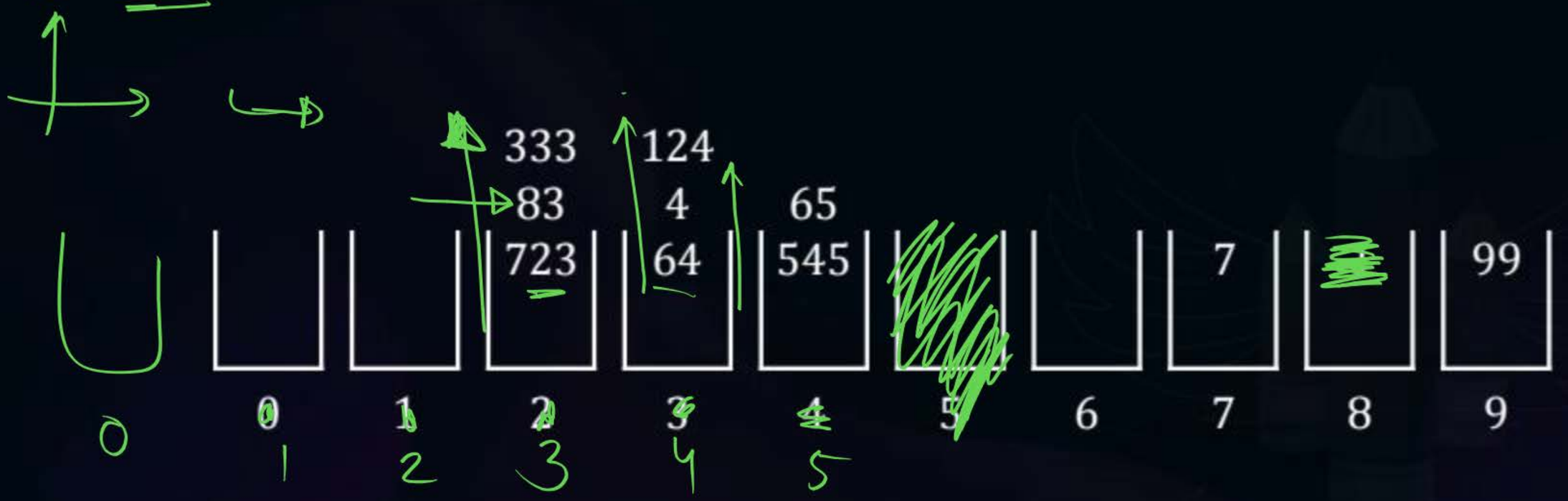
Topic : Sorting Algorithms

Pass1: Bucket as per unit place digit (Right most digit)

64, 723, 99, 83 545, 65, 333, 7, 4, 124

72 3

Pass1 : o/p : [723, 83, 333, 64, 4, 124, 545, 65, 7, 99]





Topic : Sorting Algorithms

04, 07

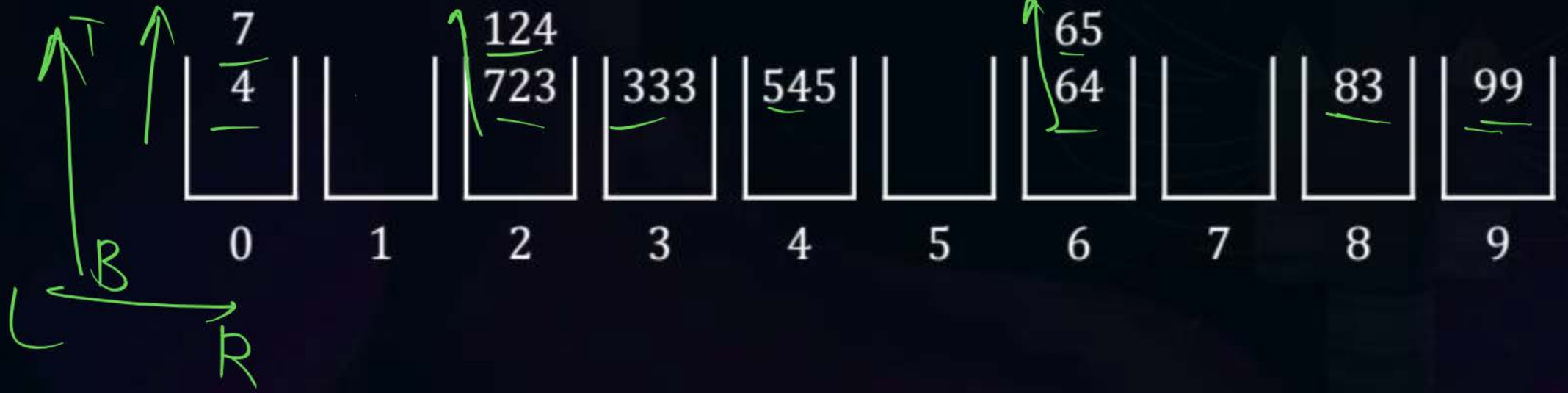
Pass2:

pass 2 → [2nd Right digit]

i/p : [723, 83, 333, 64, 4, 124, 545, 65, 7, 99]

2 8 3 6 0 2 4 6 0 9

o/p : [4, 7, 723, 124, 333, 545, 64, 65, 83, 99]





Topic : Sorting Algorithms

Pass3:

i/p : [4, 7, 723, 124, 333, 545, 64, 65, 83, 99]

o/p : [4, 7, 64, 65, 83, 99, 124, 333, 545, 723]

Final sorted o/p with total 0 comparisons.

Pass3 : 3rd digit from right





Topic : Sorting Algorithms

Note:

Time Complexity: $O(d * (n+b)) \approx O(n * d)$

b = base of given input

d = no. of digits in max elements of given i/p array

n = no. of elements in given i/p array

Radix Sort → Not inplace ⇒ SC: $O(n+K)$

→ Stable

n = no. of elems in i/p.

K = largest elem in i/p array.



Topic : Bubble Sort

#Q. What is the o/p after 3rd pass of Radix Sort?

A = [1005, 1500, 972, 555, 879, 50, 25]

No. of passes: digits(1500) = 4 passes

A = 1005, 1500, 972, 555, 879, 50, 25

(1005 → 5, 1500 → 0, 972 → 2, 555 → 5, 879 → 9, 50 → 0, 25 → 5)

0	1	2	3	4	5	6	7	8	9
1500, 50		25, 555			1005				879

P1. O/P: [1500, 50, 972, 1005, 555, 25, 879]

(1500 → 0, 50 → 5, 972 → 7, 1005 → 0, 555 → 5, 25 → 2, 879 → 7)

pass 2 :

0	1	2	3	4	5	6	7	8	9
1005, 1500		25			555, 50		879, 972		

P2 O/P: [1500, 1005, 25, 50, 555, 972, 879]

P2 O/p: [1500, 1065, 25, 50, 555, 972, 879]

5 0 0 0 5 9 8

Passes:

0	1	2	3	4	5	6	7	8	9
1065					1500			879	972
25					555				
50									

P3 O/p: [1065, 25, 50, 1500, 555, 879, 972] → Ans:-

HW



Topic : Bubble Sort

#Q. If we use Radix Sort to sort n integers in the range $(n^{k12}, n^k]$, for some $k > 0$ which is independent of n, the time taken would be:

HW

A $\Theta(n)$

B $\Theta(kn)$

C $\Theta(n \log n)$

D $\Theta(n^2)$



Topic : Bubble Sort

#Q. Give the correct matching for the following pairs :

List-I	List-II
A. O(log n)	P. Selection
B. O(n)	Q. Insertion sort
C. O(n log n)	P. Binary search
A. $O(n^2)$	P. Merge sort

HW

Codes:

A A – R B – P C – Q D – S

B A – R B – P C – S D – Q

C A – P B – R C – S D – Q

D A – P B – S C – R D – Q



Topic : Bubble Sort

#Q. If all permutations are equally likely, what is the expected number of inversions in a randomly chosen permutation of 1.....n?.

A $\frac{n(n-1)}{2}$

B $\frac{n(n-1)}{4}$

C $\frac{n(n+1)}{4}$

D $2n[\log_2 n]$

HW



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Insertion

Radix



THANK - YOU